

A Pose-based Machine Learning Approach to Video Activity Recognition

^aAndres Bueno Cardona, ^bJulio Prado, ^cJose Manuel Cardona, ^dMartín Gómez Caicedo and
^eJuan Camilo Muñoz

^aandres.bueno1909@gmail.com, ^bchepemaria2021@gmail.com, ^cjosemacar2004@gmail.com,
^dAgazazaca@gmail.com, ^eJuancamuba@outlook.com

^{a,b,c,d,e}Universidad Icesi, Faculty of Engineering and Natural Sciences,
^{a,b,c,d,e}Computer Science and Engineering Program

Universidad Icesi, Artificial Intelligence 1 course.
Santiago de Cali, Colombia.

ABSTRACT

Human Activity Recognition (HAR) from video is an enabling technology for applications in digital health, rehabilitation, sport analytics and ambient assisted living. This project develops a complete pipeline for recognizing four everyday activities—walking, turning, sitting down and standing up—from short RGB videos. Instead of training deep neural networks directly on pixels, we extract 11 body landmarks per frame using MediaPipe Pose and represent each clip as a sequence of normalized joint coordinates. From these sequences we derive kinematic features (joint angles, trunk inclination, velocities and spatial relationships) and aggregate them over short temporal windows. Using the resulting feature vectors, we train and compare three supervised classifiers: Support Vector Machines (SVM), Random Forests and XGBoost. The dataset consists of self-recorded smartphone videos from the team members under controlled conditions, with deliberate variation in camera viewpoint and clip duration. Models are tuned with cross-validated grid search and evaluated on a held-out test set using macro-averaged F1-score as the main metric. The best SVM configuration achieves a macro F1-score of 0.935 and a weighted F1-score of 0.942 on the test set, meeting the original target of 0.85. We analyze typical confusions between activities, discuss the impact of feature design and class imbalance, and outline directions for scaling the system to more complex scenarios and users.

1 Introduction

Human Activity Recognition (HAR) from video is increasingly important in areas like digital health, sports analysis and rehabilitation. Instead of working directly with raw pixels, modern approaches use human pose estimation to represent movements as sequences of body landmarks, which makes the problem lower-dimensional, more robust to appearance changes, and easier to interpret.

In this context, the project tackles the problem of building a video system that can automatically recognize everyday activities, such as walking, spin, sitting down and standing up, from short videos recorded with a cell-phone camera. This is particularly interesting because it sits at the intersection of computer vision, machine learning and human movement analysis: it enables real-time, low-

cost monitoring of activities and posture using only RGB video, with potential applications in physiotherapy, ergonomics and physical activity tracking, and it also lets us study how well such models cope with real-world factors like different viewpoints and lighting.

2 Theory

2.1 Pose-based human activity recognition

Modern pose-estimation systems such as MediaPipe Pose infer a fixed set of two- or three-dimensional body landmarks (e.g., shoulders, hips, knees, ankles) directly from RGB frames, using deep convolutional architectures trained on large annotated datasets [1, 2]. For the BlazePose model used in MediaPipe, 33 landmarks are estimated for each detected person,

including visibility scores that indicate confidence in each point [2]. The output is a skeleton-like representation that can be tracked over time to analyze motion.

Pose-based HAR uses these landmark sequences as input instead of raw pixels. At a given time t , the pose can be represented as a vector

$$\mathbf{p}_t = [x_1, y_1, z_1, \dots, x_{33}, y_{33}, z_{33}],$$

often after normalization to make the representation invariant to camera translation and scale (e.g., centering on the pelvis and scaling by shoulder distance). Over a temporal window of T frames, the clip is represented by $\{\mathbf{p}_1, \dots, \mathbf{p}_T\}$. From these, one can derive higher-level features such as joint angles, segment lengths, velocities and accelerations, which capture the kinematics of movement rather than raw positions.

2.2 Feature-based classification models

2.2.1 Support Vector Machines

Support Vector Machines (SVMs) are margin-based classifiers that try to separate classes by finding a decision boundary that leaves as much empty space as possible between examples of different classes. In the simplest (linear) case, this boundary is a hyperplane in the feature space.

When the data are not linearly separable, SVMs use a kernel function to implicitly map the original features into a higher-dimensional space where a clearer separation is possible. In practice, this is controlled by a few key hyperparameters: the regularization parameter C , which balances having a wide margin versus allowing some training errors, and the kernel parameters (for example, γ in the RBF kernel), which control how flexible the decision boundary can be.

For multi-class problems, SVMs are typically extended by training one classifier per class (one-vs-rest) or one classifier per pair of classes (one-vs-one). In this project, we use the `SVC` implementation from `scikit-learn` [4] with linear and RBF kernels, combined with feature standardization.

2.2.2 Random Forests

Random Forests are ensemble methods built from many decision trees. Each tree is trained on a slightly different random sample of the training data, and at each split only a random subset of features is considered. This randomness makes the individual trees less correlated with each other.

For classification, each tree produces a class prediction, and the forest combines them by majority vote. Because many diverse trees are averaged, Random Forests can model complex nonlinear relationships while remaining relatively robust to noise and overfitting. They also provide estimates of feature importance, which helps interpret which variables contribute most to the final decision.

2.2.3 Gradient-boosted trees (XGBoost)

Gradient boosting is another ensemble technique that also combines many decision trees, but in a sequential way. The idea is to start with a simple model and then add new trees one by one, where each new tree focuses on correcting the errors of the current ensemble.

XGBoost is an efficient and regularized implementation of gradient-boosted trees that includes several engineering improvements (such as shrinkage, column subsampling and explicit regularization) to control overfitting and speed up training [5]. In this project we use `XGBClassifier` with a small grid of hyperparameters to compare its performance against SVM and Random Forest.

2.3 Evaluation metrics

Because the dataset is moderately imbalanced across classes, overall accuracy can be misleading. We therefore report class-wise precision, recall and F1-score, and their macro and weighted averages.

For a given class, precision measures how many of the predicted examples for that class are actually correct, and recall measures how many of the true examples of that class are correctly found by the model:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

The F1-score is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

The macro F1-score averages the F1 of each class, giving equal weight to all of them, while the weighted F1-score averages them using the number of samples per class as weights. Confusion matrices complement these metrics by showing, in a single table, how often each true class is predicted as each possible label, which helps identify typical confusions between activities.

3 Methodology

The project follows an adapted CRISP-DM workflow, but here we describe only the concrete steps taken: data collection, landmark extraction and preprocessing, feature engineering, model training and evaluation, and integration into a simple real-time demo.

3.1 Dataset and data collection

The dataset consists of short RGB videos recorded by the team members. Each subject performed four activities: walking (towards and away from the camera), turning, sitting down and standing up. Videos were captured indoors with relatively simple backgrounds and uniform lighting, using smartphone cameras. The camera viewpoint and distance were varied to introduce some diversity, and clip duration was kept below approximately five seconds, resulting in at most 150 frames per video.

A first recording round revealed that some videos, especially for turning, had very few usable frames. In Deliverable 2 a second acquisition phase was carried out with better framing and more controlled durations, improving the balance and quality of the dataset. All 293 videos are stored in a Drive folder, and their metadata are stored in a Supabase PostgreSQL database, with a table for video information and another table that links each video to its pose landmarks as JSONB objects. This design allowed us to query and reuse the same data across all deliverables.

3.2 Landmark extraction and preprocessing

Pose landmarks were extracted with MediaPipe Pose for up to 150 frames per video. For each frame, we obtained a subset of 11 out of the 33 landmarks, such as `left_shoulder`, `right_shoulder`, `left_hip`, `right_hip`, `left_knee`, `right_knee`, `left_ankle`, `right_ankle`, `left_wrist`, `right_wrist` and head (calculated as the average of the ears), with (x, y, z) coordinates and a visibility score. Frames without a reliable detection were discarded. Videos with fewer than a minimum number of valid frames (around the 10th percentile, with a lower bound of 10 frames) were removed to ensure that each example contained enough temporal information.

To reduce sensitivity to camera position and subject height, we normalized the coordinates by translating the pelvis to the origin and scaling distances by a reference segment, typically the shoulder distance. A short moving average over time was applied to the landmark trajectories to smooth frame-to-frame noise from the pose estimator.

3.3 Feature engineering

From the normalized poses we computed kinematic features that describe posture and motion. These include joint angles (for example hip-knee-ankle angles and trunk inclination relative to the vertical), approximate velocities and accelerations of selected landmarks, and distances or relative positions between key joints. Features were first computed per frame, producing a multivariate time series per video.

To feed fixed-length inputs to the classifiers, we aggregated these frame-wise features over sliding temporal windows. For each window we calculated summary statistics such as mean, standard deviation, minimum and maximum for every feature. The resulting tabular dataset, where each row corresponds to a window labeled with one activity, was stored as `features.csv`. Before training SVMs, we standardized the features using z-score normalization in a scikit-learn pipeline; the same processed features were used for Random Forest and XGBoost for consistency.

3.4 Model training, validation and deployment

The feature dataset was split into training (80%) and test (20%) sets, stratified by activity, ensuring that all windows from a given video stayed in the same split. Using scikit-learn and XGBoost, we trained three models: SVM (with linear and RBF kernels), Random Forest and XGBoost. Hyperparameters were tuned with 3-fold cross-validated grid search on the training set, optimizing macro F1-score. The best configuration of each family was re-trained on the full training set and evaluated on the test set.

The SVM with RBF kernel, which obtained the highest macro and weighted F1-scores, was integrated into a simple real-time demo. Live or prerecorded video is processed frame by frame with MediaPipe, the same normalization and feature pipeline is applied online, temporal windows are formed, and the model outputs the current activity label, which is displayed on top of the video.

4 Results

This section summarizes the quantitative results obtained in Deliverable 2 from the three compared models on the held-out test set. The test set contains 35 labeled windows, with class frequencies inherited from the underlying videos (14 walking, 6 turning, 8 standing up, 7 sitting).¹

Tables 1, 2 and 3 present the class-wise precision, recall and F1-score for SVM, Random Forest and XGBoost, respectively, along with macro and weighted averages.

4.1 Support Vector Machine (SVM)

The SVM model achieves a macro F1-score of 0.935 and a weighted F1-score of 0.942. All classes obtain F1-scores above 0.90, with walking slightly higher due to perfect recall.

4.2 Random Forest

The Random Forest model achieves a macro F1-score of 0.902 and a weighted F1-score of 0.913. Performance is comparable to SVM for

walking and turning, but somewhat lower for standing up and sitting.

4.3 XGBoost

XGBoost yields metrics essentially identical to Random Forest on this dataset: macro F1-score of 0.902 and weighted F1-score of 0.913. This indicates that the chosen hyperparameter grids and dataset size do not significantly differentiate the two tree-based ensembles in terms of test performance.

4.4 Feature Reduction

To optimize the model for real-time deployment, where computational efficiency is important, we performed feature selection to identify a more compact and equally performant feature set. The analysis showed that a reduced model with only **70 selected features** (out of the original total) achieved the **same performance** as the full model: a macro F1-score of 0.887 and an accuracy of 0.914.

Further reductions in the number of features led to a gradual decline in performance, as shown below:

- **K=60 features:** F1-macro=0.861, Accuracy=0.886
- **K=30 features:** F1-macro=0.839, Accuracy=0.857
- **K=15 features:** F1-macro=0.775, Accuracy=0.829

The set of 70 selected features, which includes a mix of joint angles, velocities, spatial relationships and temporal trends, was therefore chosen for the final deployment model

5 Results analysis

The quantitative results provide several insights into the behavior of the models and the characteristics of the dataset.

¹As reported in the Spanish Deliverable 2 report.

Table 1: Classification report for SVM on the test set.

Class	Precision	Recall	F1-score	Support
walking	0.933	1.000	0.966	14
turning	1.000	0.833	0.909	6
standing_up	1.000	0.875	0.933	8
sitting	0.875	1.000	0.933	7
macro average	0.952	0.927	0.935	35
weighted average	0.948	0.943	0.942	35

Table 2: Classification report for Random Forest on the test set.

Class	Precision	Recall	F1-score	Support
walking	0.933	1.000	0.966	14
turning	1.000	0.833	0.909	6
standing_up	1.000	0.750	0.857	8
sitting	0.778	1.000	0.875	7
macro average	0.928	0.896	0.902	35
weighted average	0.929	0.914	0.913	35

5.1 Generalization and model comparison

The relatively high F1-scores across all models suggest that the feature-engineering pipeline successfully captures the key kinematic patterns distinguishing the four activities. In particular, joint angles and trunk inclination appear to be highly informative for differentiating sitting vs. standing up, while velocities and step-like patterns help distinguish walking and turning.

SVM achieves the best overall performance, with improvements most notable in the standing-up class: recall of 0.875 versus 0.750 for the tree-based models. This indicates that the SVM decision boundary in the high-dimensional feature space captures more subtle differences between standing up and other activities, likely benefiting from the standardized continuous features and the RBF kernel’s flexibility.

Random Forest and XGBoost perform very similarly, which is not surprising given the modest hyperparameter grids explored. Both achieve perfect recall for sitting but at the cost of lower precision (0.778), indicating a tendency to overpredict this class when in doubt. In contrast, SVM is more conservative for sitting (higher precision, slightly lower recall),

leading to fewer false positives at the expense of missing very few true instances.

5.2 Evidence of overfitting or underfitting

Given the dataset size, there is always a risk of overfitting, particularly for flexible models such as RBF SVMs and boosted trees. The use of 3-fold cross-validation during grid search and the reasonably close macro and weighted F1-scores across models suggest that overfitting is somewhat controlled, but the small test set (35 samples) limits the robustness of these conclusions.

The confusion patterns reported in Deliverable 2 show that most predictions fall on the diagonal, but with a few off-diagonal entries primarily between sitting and standing up. This is expected because these classes correspond to opposite transitions and share many intermediate poses. There is no strong indication of systematic misclassification to a completely unrelated class (e.g., confusing walking with sitting), which would hint at underfitting or poor feature design.

However, the absence of a completely independent subject-wise test set (where some participants are seen only in testing) means that we cannot fully assess generalization to new in-

Table 3: Classification report for XGBoost on the test set.

Class	Precision	Recall	F1-score	Support
walking	0.933	1.000	0.966	14
turning	1.000	0.833	0.909	6
standing_up	1.000	0.750	0.857	8
sitting	0.778	1.000	0.875	7
macro average	0.928	0.896	0.902	35
weighted average	0.929	0.914	0.913	35

dividuals. This is a common limitation in small HAR projects and would need to be addressed in future work.

5.3 Failure cases and limitations

The main limitations observed are:

- **Class imbalance and transitional ambiguity:** The dataset is moderately imbalanced, with more walking and turning examples. Transitional frames (e.g., the moment just before sitting down or just after standing up) can resemble each other, causing confusion between the two classes even for a human observer.
- **Limited diversity:** All participants belong to a relatively homogeneous population (university students), and videos were recorded under controlled indoor conditions with simple backgrounds. As discussed in Deliverable 1, this limits generalization to different body types, clothing, environments and camera setups.
- **Pose-estimation errors:** Although MediaPipe Pose is robust, fast motions, self-occlusions or partial views can lead to noisy landmarks. While the preprocessing pipeline includes smoothing and visibility-based filtering, some residual noise may still affect feature values and thus classification.

Despite these limitations, the system demonstrates that a relatively simple pipeline combining pose-based feature engineering with classical machine learning can achieve strong performance on a well-defined set of activities.

6 Conclusions and future work

This project implemented a complete pipeline for human activity recognition from video using pose landmarks and classical machine learning techniques. Starting from raw videos recorded by the team, the work covered database design, pose extraction with MediaPipe, exploratory analysis, feature engineering, model training and evaluation, and integration into a simple real-time demo.

The main conclusions are:

- It is feasible to distinguish between walking, turning, sitting down and standing up using only pose landmarks extracted from short RGB clips. The best SVM model achieved a macro F1-score of 0.935 and a weighted F1-score of 0.942 on the test set, surpassing the initial target of 0.85.
- Carefully designed kinematic features (joint angles, trunk inclination, velocities and spatial relationships) provide sufficient discriminative power for these activities, without the need for deep image-based models.
- Among the tested classifiers, SVM with an RBF kernel provided the best trade-off between performance and model complexity on this dataset, while Random Forest and XGBoost achieved similar but slightly lower scores.
- The main sources of error are confusions between sitting and standing up, which are structurally similar transitions, and occasional misclassifications in borderline frames. This suggests that refining how

temporal context is modeled could yield further improvements.

Several directions for future work are identified:

- **Dataset expansion:** Increase the number and diversity of participants (age, body types, clothing), record in more varied environments (different backgrounds, lighting conditions) and include more complex scenes (multiple people, activity sequences) as originally discussed in Deliverable 1.
- **Richer temporal modeling:** Replace or complement window-based feature aggregation with temporal models such as recurrent networks, temporal convolutional networks or attention-based architectures operating directly on landmark sequences.
- **Automatic segmentation:** Move from single-activity clips to continuous videos where the system must both segment and classify activities over time, possibly using tools like Label Studio for precise temporal annotation.
- **Explainability and user feedback:** Use feature importance, saliency over landmarks or rule-based explanations to provide interpretable feedback, especially in rehabilitation or sport-coaching contexts where understanding *why* an activity is classified in a certain way is important.

Overall, the project demonstrates that pose-based HAR with classical machine learning is a viable and educational approach for small-scale activity-recognition systems, and it sets a foundation for future extensions toward more realistic and demanding applications.

References

- [1] Google AI Edge, “Pose landmark detection guide,” MediaPipe Solutions, accessed Nov. 2025. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker
- [2] V. Bazarevsky, I. Grishchenko, K. Raveendran, et al., “On-device, real-time body pose tracking with MediaPipe BlazePose,” Google Research Blog, Aug. 2020. [Online]. Available: research.google/blog/on-device-real-time-body-pose-tracking-with-mediapipe-blazepose/
- [3] G. Lan, Y. Wu, F. Hu, and Q. Hao, “Vision-based human pose estimation via deep learning: A survey,” *IEEE Trans. Human-Machine Systems*, vol. 53, no. 1, pp. 253–270, Feb. 2023.
- [4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.
- [6] B. Ren, M. Liu, R. Ding, and H. Liu, “A survey on 3D skeleton-based action recognition using learning method,” *Cyborg and Bionic Systems*, vol. 5, article 0100, 2024. [Online]. Available: <https://doi.org/10.34133/cbsystems.0100>